Real Estate Triage Agent

Docker Containerization & Deployment

A comprehensive guide to packaging and deploying your AI-powered real estate ticket classification system using Docker containerization.

Made with GAMMA

Project Overview

What It Does

This AI-based real estate ticket system leverages Flask, LangChain, and Groq to automate the processing of customer inquiries. It intelligently classifies tickets, analyzes sentiment, and generates concise summaries—transforming raw customer messages into actionable insights.

Key Capabilities

- Automatic ticket classification by property type and priority
- Sentiment analysis to gauge customer情绪
- Intelligent summarization of lengthy messages
- Integration-ready for existing CRM systems

Made with GAMMA

Why Docker?

Environment Consistency

Eliminates "works on my machine" problems by packaging your application with all dependencies in a consistent environment.

Easy Deployment

Deploy your application across development, staging, and production environments with a single command.

Platform Independence

Run your application on any system—Linux, Windows, macOS—without worrying about compatibility issues.

Scalable Architecture

Scale your application horizontally by spinning up multiple containers to handle increased traffic efficiently.

Made with GAMMA

What is Docker?

Containerization Platform

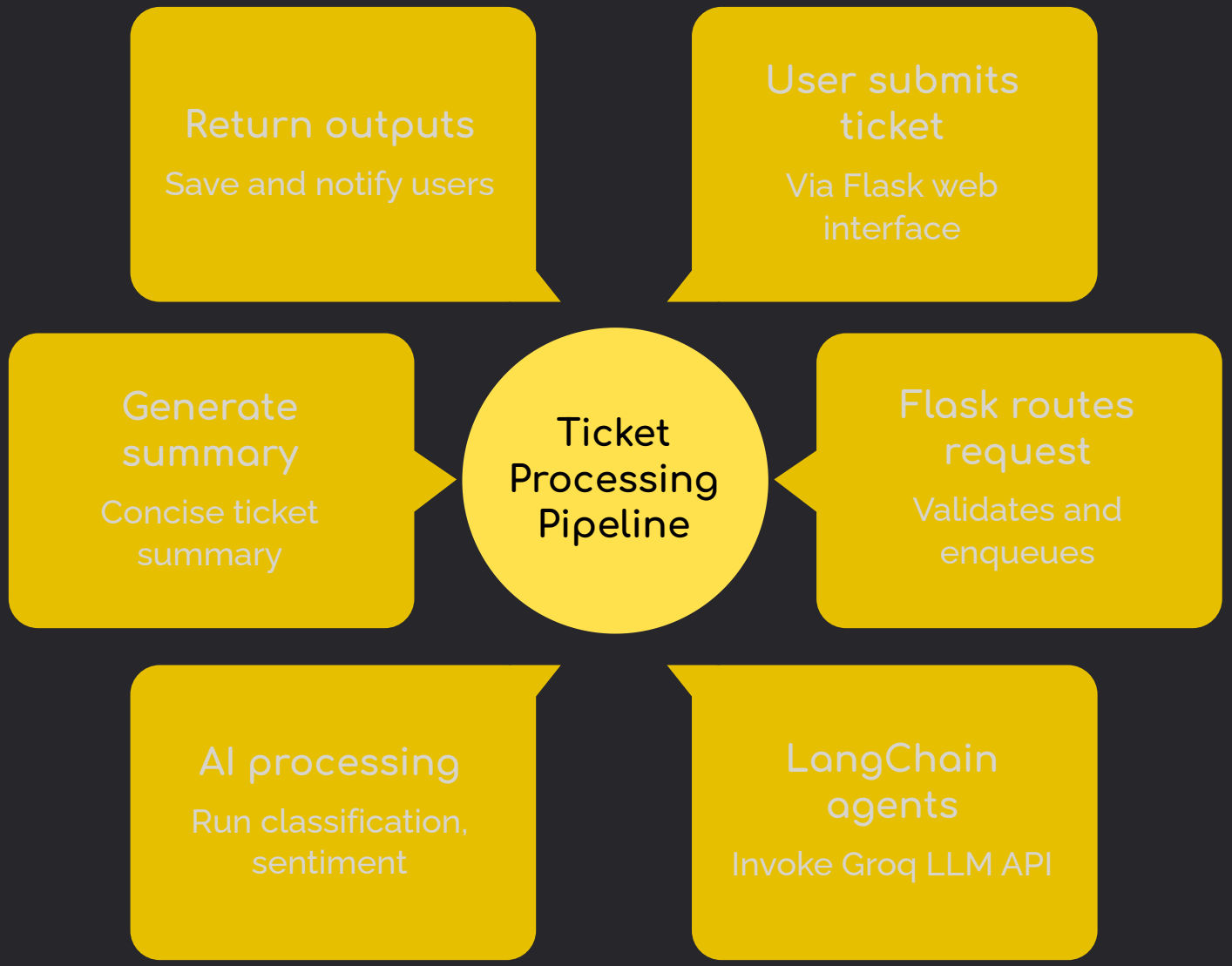
Docker is a revolutionary platform that packages applications and all their dependencies into standardized units called containers. These containers include everything needed to run your software—code, runtime, system tools, libraries, and settings.

Key Benefits

- Lightweight compared to virtual machines
- Fast startup times
- Isolated environments prevent conflicts
- Version control for your application images
- Shareable images via Docker Hub

Made with GAMMA

System Architecture



Made with GAMMA

Dockerfile

Building Your Container

The Dockerfile defines the blueprint for your container. Here's the complete file for the Real Estate Triage Agent:

```
FROM python:3.10-slim

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

EXPOSE 5000

CMD ["python", "app.py"]
```

This minimal configuration uses Python 3.10, copies your application code, installs dependencies, exposes port 5000 for Flask, and starts the application.

Made with GAMMA

Dockerfile Breakdown

01

Base Image

`FROM python:3.10-slim` - Starts with a lightweight Python 3.10 image containing Python runtime and basic tools.

02

Working Directory

`WORKDIR /app` - Sets the container's working directory to /app where your application will reside.

03

Copy Files

`COPY . .` - Copies all files from your current directory into the container's working directory.

04

Install Dependencies

`RUN pip install -r requirements.txt` - Executes pip to install all Python packages listed in requirements.txt.

05

Expose Port

`EXPOSE 5000` - Documents that the container listens on port 5000 for the Flask application.

06

Run Command

`CMD ["python", "app.py"]` - Specifies the default command to run when the container starts.

Made with GAMMA

Build Docker Image

Creating Image

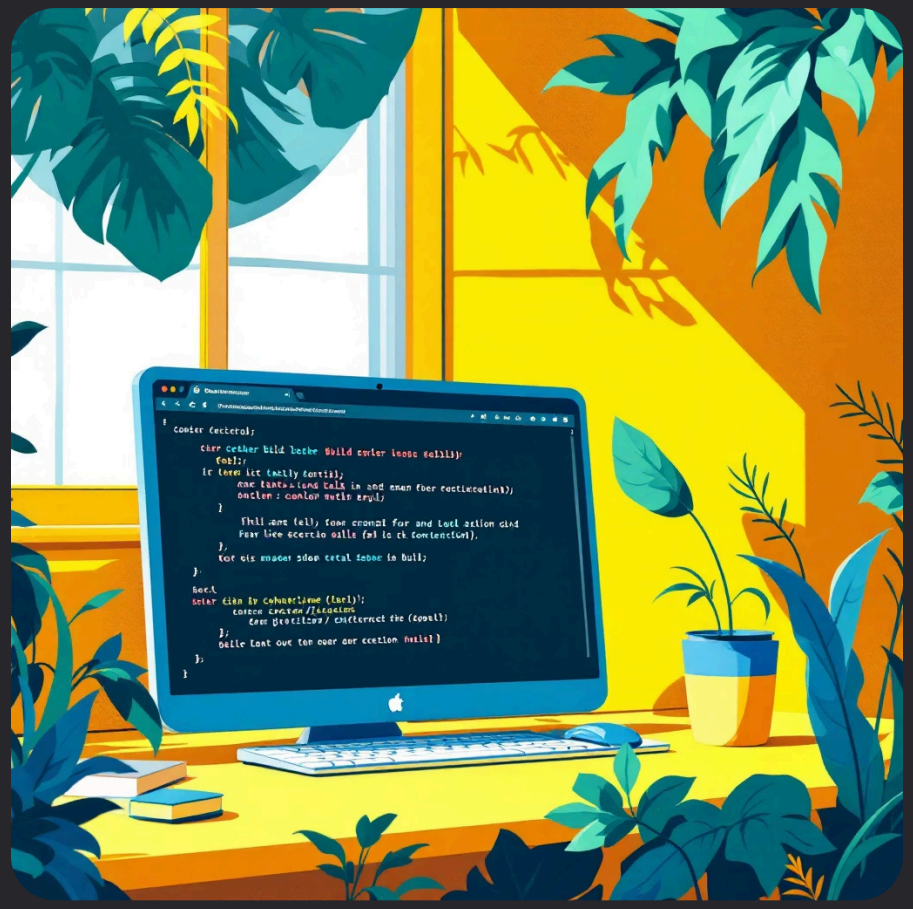
Once your Dockerfile is ready, building the image is straightforward. Navigate to your project directory in the terminal and run:

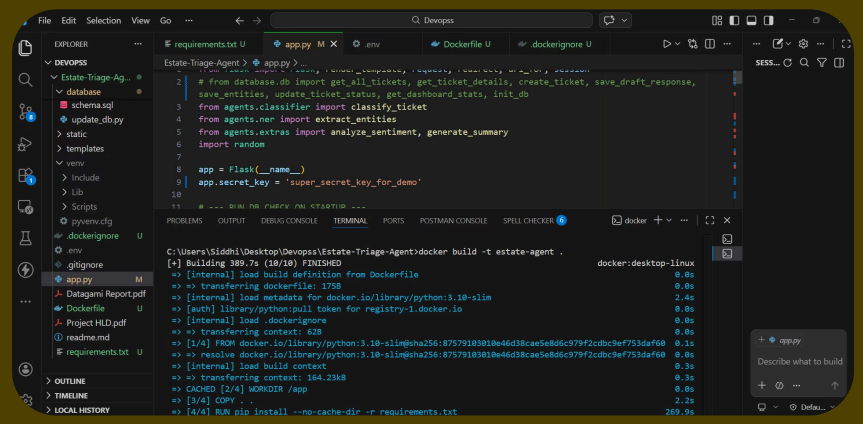
```
docker build -t estate-agent .
```

This command tells Docker to build an image using the Dockerfile in the current directory (.) and tag it with the name "estate-agent".

What Happens During Build

- Docker reads your Dockerfile instructions
- Downloads the base Python image if not cached
- Creates intermediate container layers
- Runs each command sequentially
- Creates the final image tagged "estate-agent"





Made with GAMMA

Verify Your Image

Check Image Registry

After building, verify your image exists in Docker's local registry:

```
docker images
```

This command lists all images on your system, showing repository names, tags, image IDs, creation dates, and sizes.

Expected Output

You should see an entry for `estate-agent` with a tag (usually latest), image ID, and size around 500-800MB depending on your dependencies.

Made with GAMMA

Run Your Container

Launch the Application

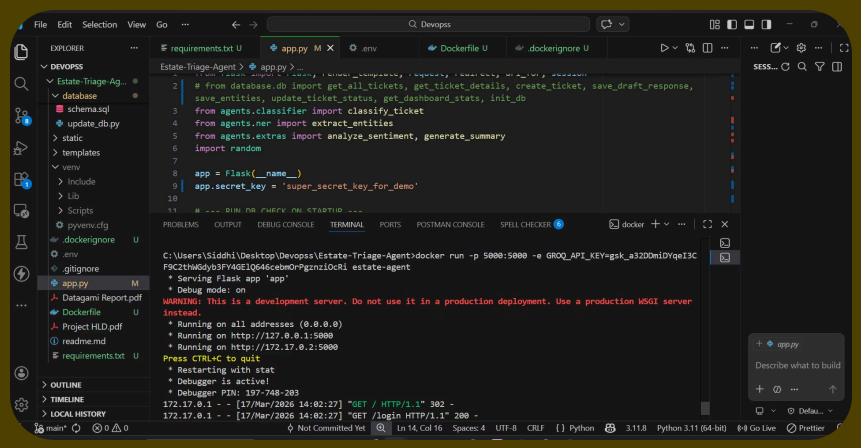
Start your container with port mapping and environment variables:

```
docker run -p 5000:5000 \
  -e GROQ_API_KEY=your_key \
  estate-agent
```

Command Breakdown

- `-p 5000:5000` - Maps container port 5000 to host port 5000
- `-e GROQ_API_KEY=...` - Passes your Groq API key as environment variable
- `estate-agent` - The image name to run

Visit `http://localhost:5000` to access your running application.



Made with GAMMA